

**Docker:** Docker is software used for OS level virtualisation. It is used to create, deploy, and run application packages called containers.

Containers allow the developer to package an application with all the parts it needs, such as libraries and other dependencies, and ship it as one package. Docker is lightweight, open and secure.

**GitHub:** This collaborative code review tool supports around 200 software languages. It also supports all the version control features of check-in, commits, branches, merging, labels, task management, wikis, push and pull to/from GitHub, etc. Git fits in very well as a popular and distributed version control system for teams located at different geographical locations.

**OpenShift:** This is Red Hat's cloud computing PaaS offering. It is an application platform in the cloud, where application developers and teams can build, test, deploy and run their applications.

**Jenkins:** Jenkins is a cloud based continuous integration tool that helps to automate the activities of build, code analysis and storing the artefacts. These activities are triggered once a developer or the team commits the code to version control repository.

Jenkins has many plugins and works as a CI tool for various technologies like C/C++, Java/J2EE, .NET, Angular JS, etc. It also provides plugins to integrate with SonarQube for code review, JFrog Artifactory for storing binary artefacts, testing tools like Selenium, etc, as a part of the automation process.

**Kubernetes:** Open source Kubernetes is free and downloaded from its repository on GitHub. Administrators must build and deploy the Kubernetes release to a local system or cluster or to a system or cluster in a public cloud, such as AWS, Google Cloud Platform (GCP) or Microsoft Azure.

**Gravitee.io:** This is an open source platform for managing APIs that are flexible and lightweight. It offers out-of-the-box features such as rate limiting, IP filtering, cross-origin resource sharing, plug-and-play options, open source API developer portal with OAuth2 and JSON web tokens policies, and load balancing.

**Tyk.io:** This open source API gateway has been developed in Go programming language. It provides an open source API developer portal, detailed documentation, a dashboard for API analytics, rate limiting for APIs, authentication, and various other such specifications that help organisations focus on microservices environments and containerisation.

**Swagger Hub:** This open source API management platform renders a wide range of options to designers and developers in the backend development domain. It provides a robust and intuitive editor that delivers higher efficiency and speed while preserving design consistency.

**MongoDB:** This is a powerful, flexible, document-oriented and scalable NoSQL database. It supports features like indexes, range queries, sorting, aggregations and geospatial indexes. It supports JSON to store and transmit information. JSON, being standard protocol, is a great advantage for both the Web and the database. MongoDB supports a rich query language to support read and write operations (CRUD) as well as data aggregation, text search and geospatial queries.

Proper implementation of microservices makes the organisation agile to changes in business and boosts time to market. The cost and time to develop and deliver applications reduces to a large extent, and systems become more resilient. Microservices bring a change in organisation communication that allows development and operations teams to formulate, strategise, and deliver in more efficient ways.

Open source tools, patterns, processes, and people play a crucial role in implementing microservices properly. These help the team to develop and manage microservices efficiently. Tools cut across design, development, deployment, tracing and monitoring. As the number of microservices grows, tools that use automation will be necessary to manage them. New processes like continuous integration and delivery, and decentralised governance need to be put in place to enable frequent releases using the microservices platform.

Finally, avoid creating too many microservices. Every new microservice uses multiple resources. Split larger services into microservices only when the demand is for scaling, life cycle, or data. Making them too small transfers complexity from the microservices into the service integration task. Also, avoid sharing microservices between systems. **END** 🐙

### Acknowledgements

The author would like to thank Santosh Shinde of BTIS, Enterprise Architecture division of HCL Technologies Ltd for giving the required time and support in many ways in bringing this article as part of architecture practice efforts.

### By: Dr Gopala Krishna Behara

The author is an enterprise architect in the BTIS Enterprise Architecture division of HCL Technologies Ltd. He has a total of 27 years of experience in the IT industry.

**Disclaimer:** The views expressed in this article are that of the author and HCL does not subscribe to the substance, veracity, or truthfulness of the said opinion.